

Lecture 11: Breaking Protections

August 26, 2024

Intro

- ▶ “Cracking” – removing protection schemes, originally from software, but really any DRM
- ▶ A series of demonstrations of breaking increasingly difficult protections
- ▶ Not to teach you to steal things, but if you intend to develop protections, you must know how crackers defeat them.
- ▶ *“Copy protection technologies developed by people who have never attempted cracking are never effective”*
- ▶ “crackme” – a contrived program to provide a puzzle to teach cracking basics

Patching

KeygenMe-3 by Bengaly

- ▶ Recognize the standard error message box
- ▶ Break on the `MessageBox` API call
- + Fundamental: find the code that denies an incorrect key. From there, it's easier to discover where the check is.
- Most developers will avoid a call to `MessageBox` and instead build their own.
- ▶ This one imports `USER32.MessageBoxA`
- ▶ Olly can search for all references to an API call. Might also find the call to the correct key message box, if one exists.

Patching

KeygenMe-3 by Bengaly

3rd reference listed by Olly:

```
0040133F CMP EAX,ESI
00401341 JNZ SHORT Key4.00401358
00401343 PUSH 0
00401345 PUSH Key4.0040348C ; ASCII "KeygenMe #3"
0040134A PUSH Key4.004034DD ; Text = " Great, You are...
0040134F PUSH 0 ; hOwner = NULL
00401351 CALL <...MessageBoxA> ; MessageBoxA

00401356 JMP SHORT Key4.0040136B
00401358 PUSH 0 ; Style = MB_OK/MB_APPLMODAL
0040135A PUSH Key4.0040348C ; Title = "KeygenMe #3"
0040135F PUSH Key4.004034AA ; Text = " You have Entered a Wrong...
00401364 PUSH 0 ; hOwner = NULL
00401366 CALL <...MessageBoxA> ; MessageBoxA

0040136B JMP SHORT Key4.00401382
```

Hand decompilation:

```
if (var_EAX == var_ESI) {
    MessageBox(NULL, "Great, You are ranked as Level-3 at Keygening now",
               "KeygenMe #3", MB_OK);
} else {
    MessageBox(NULL, "You Have Entered A Wrong Serial, Please Try Again",
               "KeygenMe #3", MB_OK);
}
```

Patching

KeygenMe-3 by Bengaly

- ▶ A simple if-else control structure.
- ▶ This looks like the right place. An easy way to test is to patch.
- ▶ Change the JNZ to either JMP or NOP. Which?
 - ▶ Consider which block you want to execute

NOTE: Cannot “insert” or “delete” instructions.

- ▶ Moving code is essentially relocating, so references must be known and fixed up.
 - ▶ More complicated than any tool is really prepared for.
 - ▶ Rather, we fill with NOPs or overwrite.
- ▶ Change to NOP and try any any values. It should always accept!

Patching

KeygenMe-3 by Bengaly

- ▶ Lesson: You may want more if statements to check the key validity.
- ▶ Lesson: You should avoid standard APIs for notifications to make the checks difficult to locate.
- ▶ Lesson: Checksums can help mitigate patching.
- ▶ You can export these changes permanently using “Copy to Executable” in OllyDbg.

Keygenning

KeygenMe-3 by Bengaly

- ▶ The challenge was to keygen, not to patch!

Goal: to mimic the key checking algorithm in order to generate as many valid keys as desired.

- ▶ Many keycheckers combine user input (e.g., name, key) with computer-specific information (e.g., primary partition volume serial)
- ▶ This prevents pirates from sharing a single key, so they need a keygen.
- ▶ For challenge response systems, the keygen must perform the vendor part (generate the key from the challenge)

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- Idea:** Locate the code that checks/generates the key, and port it and/or call it from your own program.
- ⊕ You don't need to understand the algorithm, only how to call it.
 - ▶ An fairly reliable way is to find where the program reads its input.
 - ▶ For Windows dialogs, look for the `GetDlgItemText` call or `WM_GETTEXT` message.
 - ▶ Searching for references to `GetDlgItemTextA` in Olly finds 2 calls.
 - ▶ See listings 11.1 and 11.2 in the textbook (p. 366).

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ Part of the check algorithm appears at 0040130B in Listing 11.1 (Generates a value is **ESI**)
- ▶ The other part is the entirety of Listing 11.2 (Returns a value is **EAX**)
- ▶ To “rip” the algorithm, we first need to determine its inputs and outputs
- ▶ Looking at a block of code, inputs are:
 - ▶ Consider each registers, memory location, etc.
 - ▶ Read within the block at a given address
 - ▶ Can be reached by some path starting at the first address in the block
 - ▶ Path contains no writes to the register or memory location

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ Similarly, outputs are:
 - ▶ Consider each registers, memory location, etc.
 - ▶ Written within the block at a given address
 - ▶ Last address in block can be reached from it by some path
 - ▶ Path contains no other writes to the register or memory location
- ▶ Not every “output” by this definition is necessarily used as an input later
- ▶ We’ve already identified the outputs of each block, however

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ Start with 0040130B
- ▶ **ECX** contains length of string from first input box
- ▶ The hard-coded address, 40303F contains that string
- ▶ The hard-coded address, 40351F contains an unknown string
- ▶ The string at 40303F seems to be read a DWORD at a time
- ▶ In reality, we just need to figure out what is the unknown string
- ▶ Notice that the read at 00401311 adds **EAX**, which has a value of 1 on the first iteration
- ▶ Thus, the string is actually at 403520

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ In every subsequent iteration, **EAX** is 4, so the address read is 403523
- ▶ Dumping the data at 403520 gives: (breaking execution at 00401311)
 - ▶ 00403520 25 40 24 65 72 77 72 23 %@\$erwr#
- ▶ So the program is using the values 0x25 and 0x65 (the first and fourth bytes)
- ▶ Before we go too far down this hole, we might just assume that these values are constant
- ▶ Just treat it as a blackbox that converts a username into a 32-bit number

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ The second function has a fairly basic purpose.
- ▶ It's C source might look something like this:

```
▶ DWORD Func11pt2(LPSTR pszString) { // pszString at [EBP+8]
    DWORD val = 0; // 00401394, val in EBX
    DWORD len = strlen(pszString); // 0040138E and 00401396, len in ECX
    LPSTR ptr = pszString; // 00401398
    for (; len >= 0; --len) {
        // Loop derived from LOOPD at 004013AD
        // (LOOPD decrements ECX and jmps if non-zero)
        DWORD digit = *(pszString++); // 0040139E
        digit -= '0'; // 0040139F
        for (DWORD i = len - 1; i >= 0; i++) {
            // Loop derived from LOOPD at 004013A8, i in ECX, 004013A2
            digit *= 10;
        }
        val += digit; // 004013AA
    }
    return val; // 004013AF
}
```

- ▶ It's converting a string representing a number in decimal to a 32-bit DWORD

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- So, we simply need to rip the first part from Listing 11.1 and output ESI in decimal

```
ULONG ComputeSerial(LPSTR pszString) {  
    DWORD dwLen = strlen(pszString);  
    _asm {  
        mov ecx, [dwLen] ; MS-VC inline allows reference to local variables like this  
        mov edx, 0x25 ; Force the constant presumably read by 00401311 the first time  
        mov eax, 1 ; This index is used for more than just loading that constant  
    LoopStart:  
        MOV EBX, DWORD PTR [pszString] ; Copied from 0040130B  
        mov ebx, dword ptr [ebx] ; the first loaded the *address* of pszString  
        //MOVSB EDX, BYTE PTR DS:[EAX+40351F] ; This is handled by mov edx, 0x25  
        SUB EBX, EDX  
        ... ; Copy 0040131A - 00401329  
        MOV EAX, 4  
        mov edx, 0x65 ; Force the constant presumably read by 00401311 subsequently  
        DEC ECX  
        JNZ LoopStart  
        mov eax, ESI ; return the value in ESI  
    }  
}
```

Ripping Key-Generation Algorithms

KeygenMe-3 by Bengaly

- ▶ Writing the main function ought to be easy now
- ▶ Just prompt for input or use a command line argument to get username
- ▶ Print out `ComputeSerial(username)` in decimal
- ▶ The author does this on page 369 using “John Doe”, which gives 580695444
- ▶ Trying these values in the original KeygenMe-3 passes!
- ▶ This also corroborates our assumption about the unknown string being constant

Advanced Cracking: Defender

- ▶ Created by the author to run the gamut of techniques he's imagined
- ▶ I encourage you, if you have time, to follow the walk-through in the book, or at least read through it

NOTE: You may need to use an older OS or run it in compatibility mode. It was designed for XP/2003. It may also require a single-processor system.

Advanced Cracking: Defender

Included Technologies

- ▶ Localized Function-Level Encryption - encrypts blocks of code with different keys determined at runtime
- ▶ Obfuscated API/OS Interface - uses the TEB and dynamic linking/loading to avoid any identifiable API calls
- ▶ Processor Time-Stamp Verification Thread - An additional thread to monitor the hardware for signs of debugging
- ▶ Runtime Generation of Decryption Keys - Decryption of “authorized” code requires correct inputs
- ▶ Heavy Inlining - Obfuscates program structure and makes navigation difficult

Conclusion

- ▶ Copy protections can be made very difficult
- ▶ Still only a matter of time
- ▶ Given just one valid username and key, much of the heavy lifting is done
- ▶ “Cracking” should only be used for educational purposes or to evaluate a protection
- ▶ “Crack-mes” are an excellent learning source
- ▶ You can’t design effective copy-protection measures without knowing how to break them